Application of Euclidean Distance
in
K-Nearest Neighbor Regression

Devi Prasad M

August 2026

Euclidean Distance – An Application: KNN Regression

A 4D Example: Predicting House Prices

Consider a 4-dimensional feature space where we want to predict the sale price of a house based on its characteristics.

Our features are:

- $x^{(1)}$: Living Area (in $1,000$ sq ft) – *Continuous*
- $x^{(2)}$: Distance to City Center (kms) – *Continuous*
- $x^{(3)}$: Age of Property (years) – *Integer*
- $x^{(4)}$: Number of Bedrooms – *Integer*

Our target y is the **Sale Price (in INR Crores)**.

Suppose a new house comes on the market. Our query point is a $2,000$ sq ft home, 1.0 km from the center, 10 years old, with 3 bedrooms:

$$\mathbf{x}_q = (2.0, 1.0, 10, 3)$$

Suppose our historical training dataset contains the following 12 recent sales:

Property	$x^{(1)}$ (Area)	$x^{(2)}$ (Distance)	$x^{(3)}$ (Age)	$x^{(4)}$ (Beds)	y_i (Price)
$\mathbf{x}_1$	1.5	1.5	12	3	3.5
$\mathbf{x}_2$	2.2	0.5	8	4	5.2
$\mathbf{x}_3$	2.0	1.0	9	3	4.8
$\mathbf{x}_4$	1.8	1.2	10	3	4.6
$\mathbf{x}_5$	2.5	2.0	5	4	6.0
$\mathbf{x}_6$	1.2	3.0	20	2	2.5
$\mathbf{x}_7$	2.1	0.8	11	3	4.9
$\mathbf{x}_8$	3.0	5.0	2	5	7.0
$\mathbf{x}_9$	1.9	1.5	10	2	4.1
$\mathbf{x}_{10}$	2.0	0.5	15	3	4.3
$\mathbf{x}_{11}$	2.3	1.1	10	4	5.4
$\mathbf{x}_{12}$	1.7	1.0	8	3	4.4

We choose $K = 3$.

Step 1: Measure the Euclidean Distance to Every Point

The 4-dimensional Euclidean distance to any training point $\mathbf{x}_i$ is:

$$\mathbf{dist}(\mathbf{x}_q, \mathbf{x}_i) = \sqrt{(2.0 - x_i^{(1)})^2 + (1.0 - x_i^{(2)})^2 + (10 - x_i^{(3)})^2 + (3 - x_i^{(4)})^2}$$

We calculate this Euclidean distance to *every* one of the 12 properties:

Property	y_i (Price)	$\mathbf{dist}(\mathbf{x}_q, \mathbf{x}_i)$	Property	y_i (Price)	$\mathbf{dist}(\mathbf{x}_q, \mathbf{x}_i)$
$\mathbf{x}_1$	3.5	2.12	$\mathbf{x}_7$	4.9	1.02
$\mathbf{x}_2$	5.2	2.30	$\mathbf{x}_8$	7.0	9.22
$\mathbf{x}_3$	4.8	1.00	$\mathbf{x}_9$	4.1	1.12
$\mathbf{x}_4$	4.6	0.2828	$\mathbf{x}_{10}$	4.3	5.02
$\mathbf{x}_5$	6.0	5.22	$\mathbf{x}_{11}$	5.4	1.05
$\mathbf{x}_6$	2.5	10.28	$\mathbf{x}_{12}$	4.4	2.02

$\boxed{\text{Measure: Every training point is compared with the query.}}$

Step 2: Select the K Nearest Neighbors

Since $K = 3$, we select the three properties with the smallest Euclidean distances:

Neighbor	Target y_i	$\mathbf{dist}(\mathbf{x}_q, \mathbf{x}_i)$
$\mathbf{x}_4 = (1.8, 1.2, 10, 3)$	4.6	0.2828
$\mathbf{x}_3 = (2.0, 1.0, 9, 3)$	4.8	1.00
$\mathbf{x}_7 = (2.1, 0.8, 11, 3)$	4.9	1.02

Therefore, our nearest neighborhood is:

$$N_3(\mathbf{x}_q) = \{(\mathbf{x}_4, 4.6), (\mathbf{x}_3, 4.8), (\mathbf{x}_7, 4.9)\}.$$

Step 3: Predict

The KNN regression prediction is the arithmetic mean of the target prices of these three most similar houses:

$$\hat{y}(\mathbf{x}_q) = \frac{4.6 + 4.8 + 4.9}{3} = \frac{14.3}{3} \approx 4.77$$

The KNN estimate for the unknown market value of our query house is:

$$\boxed{\text{Estimated Price } \approx \text{INR 4.77 Crores}}$$

A Critical Flaw: The Need for Feature Scaling

While the example above perfectly demonstrates the mechanics of KNN, there is a hidden mathematical flaw in applying Euclidean distance directly to raw, unscaled real-world data.

Consider the differences in scale among our features:

- A difference of 5 years in **Age** contributes $(5)^2 = 25$ to the distance sum.
- A difference of 2 kms in **Distance** contributes only $(2)^2 = 4$ to the distance sum.

Even if distance to the city center has a more massive impact on actual property value, the **Age** feature mathematically dominates the Euclidean distance simply because its numerical values are larger. In effect, we are comparing apples to oranges.

The Solution: Standardization
To prevent variables with larger scales from biasing the distance metric, distance-based algorithms require data preprocessing. We must normalize or standardize the features so they contribute equally to the geometry.

A standard approach is *standardization* or *Z-score normalization*, which transforms every feature to have a mean of 0 and a standard deviation of 1:

$$x^{(j)}_{\text{scaled}} = \frac{x^{(j)} - \mu_j}{\sigma_j}$$

where μ_j is the mean and σ_j is the standard deviation of feature j.

Z-Score Normalization and Re-computation

Step 4: Normalizing the Dataset (Z-score Normalization)

$$\mu_j = \frac{1}{N}\sum_{i=1}^{N} x_i^{(j)}, \qquad \sigma_j = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i^{(j)} - \mu_j)^2}$$

1. Step-by-Step Feature Mean and Standard Deviation Calculations

Feature 1: Living Area ($x^{(1)}$)

$$\begin{aligned}
\mu_1 &= \frac{1.5 + 2.2 + 2.0 + 1.8 + 2.5 + 1.2 + 2.1 + 3.0 + 1.9 + 2.0 + 2.3 + 1.7}{12} \\
&= \frac{24.2}{12} \\
&\approx 2.0167
\end{aligned}$$

$$\begin{aligned}
&\sum_{i=1}^{12}(x_i^{(1)} - \mu_1)^2 \\
&= (-0.5167)^2 + (0.1833)^2 + (-0.0167)^2 + (-0.2167)^2 + (0.4833)^2 + (-0.8167)^2 \\
&\quad + (0.0833)^2 + (0.9833)^2 + (-0.1167)^2 + (-0.0167)^2 + (0.2833)^2 + (-0.3167)^2 \\
&= 0.2670 + 0.0336 + 0.0003 + 0.0470 + 0.2336 + 0.6670 \\
&\quad + 0.0069 + 0.9669 + 0.0136 + 0.0003 + 0.0803 + 0.1003 \\
&= 2.4167
\end{aligned}$$

$$\begin{aligned}
\sigma_1 &= \sqrt{\frac{2.4167}{12}} \\
&= \sqrt{0.2014} \\
&\approx 0.4488
\end{aligned}$$

Feature 2: Distance to City Center ($x^{(2)}$)

$$\begin{aligned}\mu_2 &= \frac{1.5 + 0.5 + 1.0 + 1.2 + 2.0 + 3.0 + 0.8 + 5.0 + 1.5 + 0.5 + 1.1 + 1.0}{12} \\ &= \frac{19.1}{12} \\ &\approx 1.5917\end{aligned}$$

$$\begin{aligned}\sum_{i=1}^{12}(x_i^{(2)} - \mu_2)^2 \\ &= (-0.0917)^2 + (-1.0917)^2 + (-0.5917)^2 + (-0.3917)^2 + (0.4083)^2 + (1.4083)^2 \\ &\quad + (-0.7917)^2 + (3.4083)^2 + (-0.0917)^2 + (-1.0917)^2 + (-0.4917)^2 + (-0.5917)^2 \\ &= 0.0084 + 1.1917 + 0.3501 + 0.1534 + 0.1667 + 1.9834 \\ &\quad + 0.6267 + 11.6168 + 0.0084 + 1.1917 + 0.2417 + 0.3501 \\ &= 17.8892\end{aligned}$$

$$\begin{aligned}\sigma_2 &= \sqrt{\frac{17.8892}{12}} \\ &= \sqrt{1.4908} \\ &\approx 1.2210\end{aligned}$$

Feature 3: Age of Property ($x^{(3)}$)

$$\begin{aligned}\mu_3 &= \frac{12 + 8 + 9 + 10 + 5 + 20 + 11 + 2 + 10 + 15 + 10 + 8}{12} \\ &= \frac{120}{12} \\ &= 10\end{aligned}$$

$$\begin{aligned}\sum_{i=1}^{12}(x_i^{(3)} - \mu_3)^2 \\ &= (2)^2 + (-2)^2 + (-1)^2 + (0)^2 + (-5)^2 + (10)^2 \\ &\quad + (1)^2 + (-8)^2 + (0)^2 + (5)^2 + (0)^2 + (-2)^2 \\ &= 4 + 4 + 1 + 0 + 25 + 100 \\ &\quad + 1 + 64 + 0 + 25 + 0 + 4 = 228\end{aligned}$$

$$\begin{aligned}\sigma_3 &= \sqrt{\frac{228}{12}} \\ &= \sqrt{19} \\ &\approx 4.3589\end{aligned}$$

Feature 4: Number of Bedrooms ($x^{(4)}$)

$$\begin{aligned}\mu_4 &= \frac{3+4+3+3+4+2+3+5+2+3+4+3}{12} \\ &= \frac{39}{12} \\ &= 3.2500\end{aligned}$$

$$\begin{aligned}\sum_{i=1}^{12}(x_i^{(4)} - \mu_4)^2 \\ &= (-0.25)^2 + (0.75)^2 + (-0.25)^2 + (-0.25)^2 + (0.75)^2 + (-1.25)^2 \\ &\quad + (-0.25)^2 + (1.75)^2 + (-1.25)^2 + (-0.25)^2 + (0.75)^2 + (-0.25)^2 \\ &= 0.0625 + 0.5625 + 0.0625 + 0.0625 + 0.5625 + 1.5625 \\ &\quad + 0.0625 + 3.0625 + 1.5625 + 0.0625 + 0.5625 + 0.0625 \\ &= 8.2500\end{aligned}$$

$$\begin{aligned}\sigma_4 &= \sqrt{\frac{8.2500}{12}} \\ &= \sqrt{0.6875} \\ &\approx 0.8292\end{aligned}$$

2. Normalizing the Query Point $\mathbf{x}_q$

Using our calculated parameters (μ_j, σ_j) with $N = 12$, we standardize the query vector $\mathbf{x}_q = (2.0, 1.0, 10, 3)$:

$$x_{q,\text{scaled}}^{(1)} = \frac{2.0 - 2.0167}{0.4488} \approx -0.0372$$

$$x_{q,\text{scaled}}^{(2)} = \frac{1.0 - 1.5917}{1.2210} \approx -0.4846$$

$$x_{q,\text{scaled}}^{(3)} = \frac{10 - 10}{4.3589} = 0$$

$$x_{q,\text{scaled}}^{(4)} = \frac{3 - 3.2500}{0.8292} \approx -0.3015$$

$$\mathbf{x}_{q,\text{scaled}} = (-0.0372, -0.4846, 0, -0.3015)$$

3. Standardized Feature Matrix & Updated Distances

Applying $x_{\text{scaled}} = \frac{x-\mu}{\sigma}$ to the dataset produces the standardized feature matrix and recalculated Euclidean distances to $\mathbf{x}_{q,\text{scaled}} = (-0.0372, -0.4846, 0, -0.3015)$:

Property	$x^{(1)}_{\text{scaled}}$	$x^{(2)}_{\text{scaled}}$	$x^{(3)}_{\text{scaled}}$	$x^{(4)}_{\text{scaled}}$	y_i (Price)	$\textbf{dist}(\mathbf{x}_{q,\text{scaled}}, \mathbf{x}_{i,\text{scaled}})$
$\mathbf{x}_1$	−1.1512	−0.0751	0.4588	−0.3015	3.5	1.2725
$\mathbf{x}_2$	0.4084	−0.8941	−0.4588	0.9045	5.2	1.4252
$\mathbf{x}_3$	−0.0372	−0.4846	−0.2294	−0.3015	4.8	**0.2294** (Rank 1)
$\mathbf{x}_4$	−0.4828	−0.3208	0	−0.3015	4.6	**0.4748** (Rank 3)
$\mathbf{x}_5$	1.0768	0.3344	−1.1471	0.9045	6.0	2.1638
$\mathbf{x}_6$	−1.8197	1.1534	2.2942	−1.5075	2.5	3.5466
$\mathbf{x}_7$	0.1857	−0.6484	0.2294	−0.3015	4.9	**0.3594** (Rank 2)
$\mathbf{x}_8$	2.1910	2.7912	−1.8353	2.1105	7.0	4.9882
$\mathbf{x}_9$	−0.2600	−0.0751	0	−1.5075	4.1	1.2930
$\mathbf{x}_{10}$	−0.0372	−0.8941	1.1471	−0.3015	4.3	1.2180
$\mathbf{x}_{11}$	0.6313	−0.4027	0	0.9045	5.4	1.3813
$\mathbf{x}_{12}$	−0.7056	−0.4846	−0.4588	−0.3015	4.4	0.8107

4. Re-evaluating $K = 3$ Neighbors and Final Prediction

The 3 nearest neighbors under the standard deviation ($N = 12$) normalization are:

Neighbor	Target y_i	$\textbf{dist}(\mathbf{x}_{q,\text{scaled}}, \mathbf{x}_{i,\text{scaled}})$
$\mathbf{x}_3$	4.8	0.2294
$\mathbf{x}_7$	4.9	0.3594
$\mathbf{x}_4$	4.6	0.4748

The standardized KNN regression prediction yields:

$$\hat{y}(\mathbf{x}_q) = \frac{4.8 + 4.9 + 4.6}{3} = \frac{14.3}{3} \approx 4.77 \text{ Crores}$$

Key Insights

Why Feature Scaling Matters

1. **Equalizing Feature Weight (Standardization):** Using standard deviation ($\sigma = \sqrt{\frac{1}{N}\sum(x_i - \mu)^2}$) to rescale each feature column ensures that, after transformation, that feature has standard deviation 1 across the dataset. This prevents higher-magnitude attributes (such as property age in years) from dominating the distance metric purely due to measurement units.

2. **Shift in Neighborhood Rankings:**

   - **Unstandardized Top Neighbor:** $\mathbf{x}_4$ (dist $= 0.2828$), selected primarily because its property age matched $\mathbf{x}_q$ perfectly (10 years), masking significant differences in living area and distance.
   - **Standardized Top Neighbor:** $\mathbf{x}_3$ (dist $= 0.2294$), which moves to Rank #1 because all 4 features contribute proportionally once mean-centered and scaled by σ.

3. **Generalization Rule:** Test/query points must always be normalized using the **training set** parameters ($\mu_{\text{train}}, \sigma_{\text{train}}$), never parameters recomputed from the test data — doing so causes information leakage.